

Tarea 1

Detección y tracking de objetos basado en el modelo de Hubel y Wiesel

Profesor: Claudio Pérez Flores

Auxiliar: Christian Díaz Guerra

Ayudante: Renato Cariaga Cárcamo

Ayudante de Laboratorio: Juan Pablo Pérez Caballero

1. Objetivo

El objetivo de esta tarea es desarrollar un sistema de detección de objetos basado en filtros convolucionales (inspirado en el modelo de Hubel y Wiesel), lidiar con la supresión de detecciones redundantes y extender estos conceptos para resolver problemas de lógica espacial en tableros de juego. Finalmente, se extenderá este modelo a la extracción de características mediante Redes Neuronales Convolucionales (CNNs) aplicadas al tracking en video.

2. Descripción

El reconocimiento de objetos es un problema altamente desafiante debido a la alta variabilidad de patrones. En la actualidad, las CNNs han presentado un excelente desempeño gracias a la extracción de características por medio de filtros convolucionales. Esta tarea explora desde la creación manual de filtros para detectar símbolos estáticos, hasta el uso de filtros pre-entrenados para mantener el seguimiento de un objeto en movimiento.

En todas las secciones, **el análisis de los resultados es tan importante como la implementación misma.**

3. Detección básica de un patrón

En esta sección, se realiza la detección de un patrón simple dentro de una grilla de símbolos. Esta aproximación servirá como base para entender los conceptos básicos de los detectores basados en convolución.

3.1. Implementación de la convolución

Implemente una función que reciba como entrada un kernel (filtro) y una imagen, y que realice la convolución entre ambos. Asegúrese de que su función pueda manejar cualquier tamaño de kernel e imagen, y que la imagen de salida mantenga las dimensiones de la imagen original. Resuelva el problema de los bordes utilizando alguna técnica de *padding* (ej. zero padding).

3.2. Implementación del filtro detector

Utilizando la imagen **grid_symbols.png**, conviértala a imagen binaria utilizando un umbral adecuado a partir de la imagen en escala de grises. Extraiga el recorte del símbolo de exclamación (figura 1). Para ello, puede utilizar los archivos **utils** para encontrar las coordenadas necesarias. Deje escrito explícitamente qué coordenadas utilizó.

Luego, cree un filtro detector de este patrón, siguiendo el esquema de Hubel y Wiesel. Para ello, utilice un kernel de 3x3 balanceado (-1 en los bordes, 8 en el centro) y haga la convolución entre este kernel y la imagen binaria del símbolo extraído, lo que le dará un filtro de template matching. Muestre la imagen resultante.



Figura 1: Símbolo de exclamación a extraer

3.3. Detección del símbolo en la grilla

Utilizando el filtro de template matching creado, aplique la convolución con la imagen **grid_symbols.png** binarizada para detectar la presencia del símbolo de exclamación en la grilla. Presente la imagen resultante utilizando un mapa de calor (heatmap) para visualizar las activaciones y explique los resultados obtenidos.

3.4. Determinación del umbral de detección

A partir de la imagen resultante de la convolución, determine un umbral adecuado para identificar las posiciones donde el símbolo de exclamación está presente. Para ello, convierta la matriz de activaciones en un vector unidimensional y analice su distribución. Justifique la elección del umbral. Un ejemplo de este procedimiento se muestra en la figura 2.

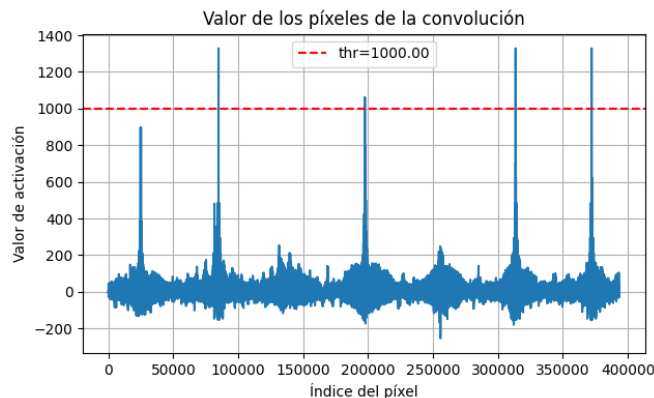


Figura 2: Determinación del umbral de detección

A partir de este umbral, grafique sobre la **imagen original** las posiciones donde se detecta el símbolo de exclamación utilizando un *bounding box* y muestre la cantidad de detecciones, como se ejemplifica en la figura 3.

HINT: El tamaño del *bounding box* se relaciona con el tamaño del filtro utilizado para la detección.



Figura 3: Bounding boxes de detecciones

3.5. Non-maximum supression (NMS)

Utilice un umbral más bajo de detección (e.g., la mitad del utilizado anteriormente). Notará que un mismo símbolo genera múltiples activaciones cercanas (omite el problema de que se detectan otros símbolos). Para resolver este problema, implemente un algoritmo de *Non-Maximum Suppression* (NMS) que conserve únicamente el píxel con la máxima activación dentro de una vecindad correspondiente al tamaño del objeto. Aplique este algoritmo sobre la imagen de activaciones y grafique los resultados finales sobre la imagen original utilizando bounding boxes.

3.6. Análisis del impacto del ruido

Manteniendo el filtro creado en la sección 3.2 y el umbral determinado en la sección 3.4, aplique el mismo procedimiento sobre la imagen `grid_symbols_blurred.png`. Analice cómo el ruido de la imagen afecta las activaciones. ¿Pudo detectar el símbolo de exclamación? Baje el umbral para intentar detectar el símbolo. ¿Pudo detectar todas sus instancias? ¿Cuántos falsos positivos obtuvo? ¿Este método es robusto ante el ruido? Justifique sus respuestas.

4. Aplicación a tableros de juego

Utilizando el sistema de detección y NMS desarrollado, resuelva los siguientes casos de estudio.

4.1. Blackjack: Conteo de manos

Utilizando la imagen `blackjack.png`, realice lo siguiente:

1. Extraiga el símbolo de diamante de alguna de las cartas y cree un filtro detector para este patrón.
2. Aplique su filtro sobre la imagen del tablero para detectar los diamantes en la imagen.
3. Ajuste el umbral de detección para lograr cubrir la mayor cantidad de diamantes posibles, sin generar demasiados falsos positivos.
4. La mesa está dividida en 4 cuadrantes claramente definidos (uno para cada jugador). Utilizando las coordenadas de sus detecciones, asigne cada carta a un jugador y cuente cuantos diamantes detectó. Calcule el valor total de la mano de cada uno. Utilice la versión sin NMS y con NMS para comparar los resultados.

Análisis: ¿Pudo detectar correctamente la cuenta de cada jugador? ¿Qué pasaría si las cartas se repartieran en un ángulo o quedaran parcialmente ocluidas? ¿Qué sucedería si se utilizan cartas de otro tipo, por ejemplo, cartas de trébol? Proponga soluciones para cada uno de estos casos. No es necesario implementar estas soluciones, solo describirlas conceptualmente.

4.2. Dominó: Validación visual

Dada la imagen **tablero_domino.png** y recortes de las **mitades** de las fichas, que se encuentran en la carpeta **fichas_domino/**, realice lo siguiente:

1. Cree un filtro detector para cada una de las mitades (1, 2, 3, 4, 5 y 6).
2. Aplique cada filtro sobre la imagen del tablero para detectar las posiciones de cada número. Puede que necesite ajustar el umbral de detección para cada número. Para ello, considere que conoce la cantidad de veces que aparece cada número en el tablero (ej. el número 1 aparece 3 veces, el número 2 aparece 4 veces, etc.), por lo que puede ordenar las activaciones de mayor a menor y elegir el umbral que permitiría detectar la cantidad correcta de cada número.
3. Utilizando las coordenadas de las detecciones, pinte el bounding box de cada número con un color distinto (ej. rojo para los 1, azul para los 2, etc.) para visualizar la distribución de los números en el tablero.

Análisis: ¿Pudo detectar correctamente las fichas? ¿Existieron detecciones falsas? Las fichas de dominó se colocan tanto en formato vertical como horizontal. Dado que el *template matching* clásico no es invariante a la rotación, ¿puedo lograr encontrar el número 3 y 2 sin tener tantos falsos positivos? Proponga una solución conceptual clara para mejorar este detector de acuerdo con los problemas planteados anteriormente. No es necesario implementar esta solución, solo describirla conceptualmente.

5. Tracking en video y Filtros CNN Pre-entrenados

En esta sección llevaremos la detección al dominio del video para realizar *tracking*. Para realizar las convoluciones, se recomienda utilizar **cv2.filter2D**, que permite realizar convoluciones 2D de manera eficiente.

Se le proporciona un clip de video del videojuego Galaga **galaga.mp4**. El objetivo es seguir al jugador a lo largo de los frames. En esta sección se trabajará con imágenes en **escala de grises**. Para ello, realice lo siguiente:

1. Extraiga los frames del video. Para ello, puede utilizar la función **cv2.VideoCapture** de OpenCV o cualquier otra librería que permita leer videos y extraer sus frames.
2. Extraiga el *template* del jugador en el frame 1 como se muestra en la figura 4.

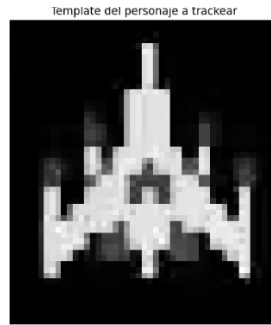


Figura 4: Personaje del videojuego Galaga

3. Utilizando PyTorch o Keras, cargue un modelo pre-entrenado (ej. VGG16) y extraiga los filtros de la **primera capa convolucional**.

HINT: Investigue cómo acceder a los tensores de la primera capa según su librería (ej. `modelo.features[0].weight.data` en PyTorch). Revise con cuidado las dimensiones de la matriz extraída utilizando `.shape` para asegurarse de promediar el eje correcto correspondiente a los canales RGB, sin alterar la cantidad de filtros ni su tamaño espacial.

4. Agregación 1 (Adaptación a grises): Promedie los 3 canales de profundidad de los pesos de la red. Esto convertirá los filtros RGB en N filtros 2D convencionales, que llamaremos $\mathbf{F}$.

5. Convolucione estos N filtros $\mathbf{F}$ con el template del jugador para obtener N template de características $\mathbf{T}$.

6. Forme un banco de filtros utilizando solo aquellos que mejor respondan a su personaje. Para ello, calcule la suma del valor absoluto de cada template de características $\mathbf{T}$ y seleccione los 5 filtros con mayor respuesta, que llamaremos $\mathbf{T}_b$. Además, guarde los 5 filtros de la CNN que los produjeron, los que formarán $\mathbf{F}_b$. Muestre estos 5 filtros seleccionados y sus respectivos template de características.

7. Realice la convolución de cada filtro de $\mathbf{F}_b$ con el primer frame del video. Obtendrá 5 mapas de características.

8. Convolucione cada template de $\mathbf{T}_b$ de su banco de filtros sobre su respectivo mapa de características del paso anterior (siga el mismo orden, i.e., primero con primero, segundo con segundo, etc.). Obtendrá 5 mapas de activación.

9. Agregación 2 (Colapso de mapas): Combine estos 5 mapas de activación en un único saliency map final. Para ello, calcule el valor máximo píxel a píxel a través de las 5 activaciones.

10. Utilizando el resultado en este frame, encuentre el umbral de detección adecuado para este mapa de prominencia y aplique un algoritmo de NMS para detectar la posición del jugador en el frame. Dibuje un bounding box alrededor de las posiciones detectadas.

11. Repita el proceso desde el paso 7 al 10 para cada frame del video, utilizando la misma configuración de umbral encontrado anteriormente, y guarde el video resultante (con los frames originales a color) con los bounding boxes dibujados. Para ello, puede utilizar la función `cv2.VideoWriter` de OpenCV o cualquier otra librería que permita escribir videos a partir de frames. En el video, podría ser útil mostrar el número de frame actual para facilitar el análisis posterior, utilizando la función `cv2.putText` o similar.

Análisis: Explique conceptualmente la idea de utilizar filtros pre-entrenados para el tracking. ¿Por qué podrían ser más robustos que un filtro clásico creado manualmente? Identifique los frames exactos donde el tracking falló (falsos negativos o pérdida del personaje). ¿Por qué falló? ¿Cómo mejoraría los resultados?

6. Entregables

1. **Reporte (PDF):** Un documento con sus explicaciones y análisis de cada sección. Este reporte debe incluir las imágenes solicitadas.
2. **Video:** El archivo **.mp4** resultante del tracking con CNN.
3. **Código Fuente:** Scripts o Jupyter Notebooks documentados. Asegúrese de que las rutas a las imágenes sean relativas para que el equipo docente pueda ejecutar su código. El código debe poder ser ejecutado sin mayores cambios ni errores, y debe generar los resultados presentados en el reporte.

Fecha de entrega: 6 de mayo a las 23:59 hrs mediante U-Cursos.